CSC3060 Project 5 Report

Student Name Chaoyi, Sun
Student ID 124090550

1 Data Parallelism

The `naive_bitwise` performs a custom bit-mixing transformation on each pair of 8-bit integers. For input bytes A and B, with constants $L = \texttt{0x5A}$ and $R = \texttt{0xC3}$:

$$\begin{aligned}\text{mixed0} &= \big((A \oplus B) \wedge L\big) \ \vee \ \big(\neg(A \wedge B) \wedge \neg L\big) \\ \text{mixed1} &= \big((A \vee B) \oplus R\big) \wedge \big((A \wedge B) \vee \neg R\big) \ \oplus \ (A \oplus B) \\ \text{result} &= \text{mixed0} \oplus \text{mixed1}\end{aligned}$$

Since all bitwise operations are per-byte independent, we pack 8 consecutive `int8_t` values into a `uint64_t` and compute on all 8 bytes in parallel using standard bitwise operators. This achieves 8× data-level parallelism and reduces instruction count. The remaining bytes are handled with scalar computation.

2 Branch Optimization

We eliminate branches using the arithmetic identity:

$$\max(x, 0) = \frac{x + |x|}{2} = \begin{cases} x, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

This formulation replaces the branch with branchless operations, completely avoiding misprediction penalties. Additionally, 8-way loop unrolling reduces loop overhead and improves instruction-level parallelism.

In practice, `std::max(0, x)` is even faster as it compiles to a single `MAXSS` instruction, achieving the same branchless behavior with fewer operations. However, we present $\frac{x+|x|}{2}$ here to clearly illustrate the arithmetic transformation that eliminates branches.

3 Cache Optimization

3.1 Matrix Multiplication

To mitigate cache misses, we apply **tiling** with a block size of $B = 256$. Tiling partitions the matrices into 256×256 submatrices. These active blocks fit comfortably within the fast L1/L2 cache hierarchy, maximizing data reuse before eviction. This theoretically reduces the cache miss complexity from $\mathcal{O}(n^3)$ to $\mathcal{O}\left(\frac{n^3}{B}\right)$. The computation is partitioned as:

$$C_{ij} = \sum_{k=0}^{n-1} A_{ik}B_{kj} \quad \Rightarrow \quad C_{IJ} = \sum_{K} A_{IK} \times B_{KJ}, \text{ where } I, J, K \text{ denote the partitioned block indices}$$

Within each block, we perform loop permutation, reordering the nested loops from the standard i-j-k sequence to i-k-j. Because matrix multiplication relies on scalar addition and multiplication, which are mathematically associative and commutative, accumulating the partial sums in a different order strictly preserves the final result.

$$C_{ij} = \sum_k A_{ik} B_{kj} \quad \xrightarrow{\text{reordered as}} \quad C_{i,*} \mathrel{+}= \sum_k \Big(A_{ik} \times B_{k,*} \Big)$$

3.2 Trace Replay

The baseline accesses `records[trace[i]]` with random indices, defeating hardware prefetching. Each access suffers a cache miss, causing the processor to stall on memory latency.

The optimized implementation uses software prefetching via the `__builtin_prefetch` intrinsic. The solution employs double prefetching with distance 64 and two future indices are prefetched simultaneously, keeping multiple memory requests in flight. This overlaps computation with memory access more effectively, reducing stall cycles. The distance of 64 is tuned to balance timely data arrival against cache pollution on the course server.

4 Optimized Data Structure

4.1 Graph

Converting the pointer-based adjacency list to CSR format stores all edges in a contiguous array `col_idx` with `row_ptr` marking node boundaries:

$$\texttt{row\_ptr} = [0,\ d_0,\ d_0 + d_1,\ \ldots,\ |E|], \qquad \texttt{col\_idx} = [\text{neigh}(0),\ \text{neigh}(1),\ \ldots]$$

Traversal for node u becomes a linear scan over $\texttt{col\_idx}[\texttt{row\_ptr}[u] : \texttt{row\_ptr}[u+1]]$. This eliminates pointer chasing and enables hardware prefetchers to load future cache lines in advance, significantly reducing cache misses.

These transformations replace random pointer chasing with predictable sequential access, significantly reducing cache misses.

4.2 Filter Gradient

The baseline stores each channel in a separate `std::vector<float>`, requiring nine independent arrays. Accessing all channels of a pixel involves nine scattered memory loads.

The optimization packs the nine channel values into a single `struct`, stored contiguously in one `std::vector<Pixel>`. A pixel's channels now reside in consecutive memory, allowing a single load to bring all nine values into the cache. This reduces memory accesses from nine per pixel to one, significantly improving spatial locality and cache efficiency.

5 Bit Manipulation

The baseline Black-Scholes kernel calls `std::sqrt`, `std::log`, and `std::exp` for each option price. These standard library functions are accurate to 1 ULP but introduce unnecessary overhead, as the checker only requires a relative tolerance of 5×10^{-3}.

The optimization replaces these functions with fast approximations using bit-level manipulation:

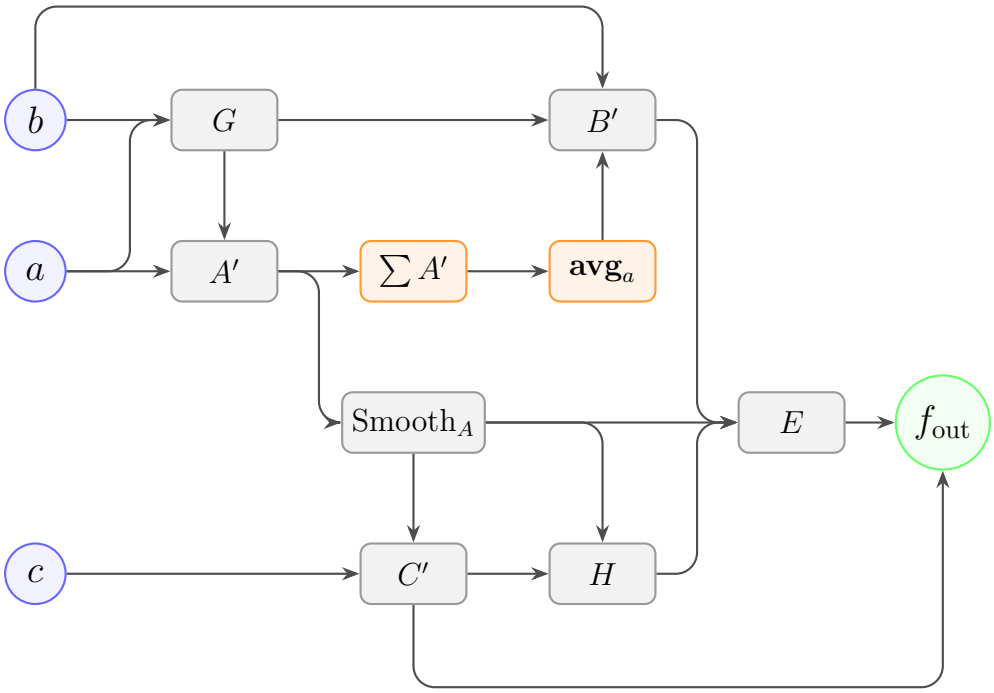
- **Square root** Fast inverse square root with Newton iteration. The method first approximates the reciprocal square root $\frac{1}{\sqrt{x}}$ using integer bit operations on the floating-point representation, then obtains $\sqrt{x} = x \cdot \frac{1}{\sqrt{x}}$.
- **Exponential** Using the identity $e^x = (1 + \frac{x}{n})^n$ with $n = 256$, the approximation is computed by repeated squaring.

6 Sparse Matrix Optimization

The baseline implementation repeatedly loads the same CSR indices and values for each dense column, and accesses the dense matrix with a large stride, causing frequent cache misses.

The optimization applies two techniques. First, the loop order is interchanged so that each non-zero element is loaded once and reused across all dense columns. Second, the dense matrix is transposed from [dense_cols][cols] to [cols][dense_cols], turning strided accesses into sequential stream accesses and improving cache efficiency.

7 Loop Fusion



The baseline uses nine sequential loops, each writing intermediate results to memory and reloading them later, causing excessive memory traffic.

The data dependencies show that avg_a requires the sum of all A', forcing at least two passes. The optimization restructures the computation into two passes:

- Compute A' and accumulate $\sum A'$.
- With avg_a known, fuse Stages 4 $\sim$ 9 into a single loop, keeping all intermediates in registers.

8 Function Inline

8.1 Function Inlining

The baseline calls seven functions per pixel, each with `noinline` attribute forcing call overhead and preventing cross-function optimizations.

The optimization manually inlines small functions where call overhea dominates computation: `apply_gain`, `apply_shift`, `apply_limit`, `color_correct`, `compute_gray`, `enhance_contrast`, and `calculate_gain`/`hdr_compress`. Large functions with complex logic (`complex_mask_logic`, `importance_weight`) remain as calls to avoid code bloat.

This eliminates six function calls per pixel, enables constant propagation and common subexpression elimination, and allows the compiler to vectorize the inner loop.

9 Bonus

Use the parameters `-O3 -march=native -fno-associative-math -fno-reciprocal-math -fno-trapping-math` in the `CMakeLists.txt`.

```
set_source_files_properties(
    ${CMAKE_SOURCE_DIR}/src/kernel/matmul.cpp
    PROPERTIES COMPILE_FLAGS "-O2 -fno-tree-vectorize"
)
```

Also add the above to avoid floating-point precision discrepancies caused by compiler auto-vectorization. Since single-precision floating-point arithmetic is non-associative, altering the accumulation order via SIMD instructions leads to minor numerical deviations, which would trigger false positives in the strict relative error checker.

To compensate for the performance penalty incurred by disabling SIMD vectorization, thread-level parallelism is introduced via OpenMP. Specifically, by applying the `#pragma omp parallel for` directive to the outermost loop, the computational workload is distributed across multiple CPU cores. This allows independent matrix rows to be computed concurrently, achieving massive speedups without altering the strictly required thread-local accumulation order. To seamlessly integrate OpenMP at the build-system level, the following configuration is appended to the `CMakeLists.txt`:

```
find_package(OpenMP REQUIRED)
target_link_libraries(kernels PUBLIC OpenMP::OpenMP_CXX)
```

10 Results

Table 1: Performance Comparison: Baseline vs. Basic vs. Bonus

Benchmark	**Default Input Size**	**Baseline (ns)**	**Basic**		**Bonus**	
			Time (ns)	**Speedup**	**Time (ns)**	**Speedup**
Black-Scholes	81,920 options	4,800,000	3,817,671	1.257x	3,299,603	1.455x
Sparse SpMM	2048×2048 for S and D^T	116,000,000	79,345,670	1.462x	55,047,604	2.107x
ReLU	vector length 1,024,000	550,000	394,952	1.393x	379,915	1.448x
Bitwise	vector length 1,024,000	250,000	240,909	1.038x	239,224	1.045x
MatMul	matrix size 512×512	88,000,000	77,836,484	1.131x	47,984,715	1.834x
Trace Replay	65,536 records, trace length 1,048,576	3,400,000	3,226,289	1.054x	3,027,006	1.123x
Graph	1,024,000 nodes, avg. degree 8	5,000,000	5,250,533	0.952x	3,328,963	1.502x
GRFF	feature size 1,024,000	8,500,000	8,855,230	0.960x	3,312,860	2.566x
Image Proc	image size 1024×1000	43,000,000	42,494,877	1.012x	1,916,674	22.435x
Filter Gradient	1024×1024 for height $\times$ width	25,000,000	19,713,140	1.268x	1,295,622	19.296x
Geometric Mean	-	-	-	**1.140x**	-	**2.632x**

11 Appendix

It is hereby declared that Generative AI tools were utilized in this project to assist with code optimization, report polishing, and test case generation. The core logic and implementation were verified by the author.